Jenni: Knowledge Transfer (KT) Document

Jenni is a mobile app (and web build) for pet owners in India. Pet owners find and book **veterinarians** (clinic visits) and **pet-care businesses** (grooming, boarding, training), pay an advance, and keep their pets' records. Vets and pet-care businesses run their work from the same app, and an **admin** approves them before they go live.

This document explains the whole code base, the way it is put together, and every important flow from the first tap to the database row. It is written for a developer who is taking the project over.

Backend	Java 21, Spring Boot 3.3.4, PostgreSQL, Flyway (migrations V1 to V22)
Frontend	Expo SDK 58, React Native 0.88, TypeScript, React Navigation
Repository	`backend/`, `frontend/`, `docs/`, `docker-compose.yml`
Older docs	`README.md` (setup), `docs/PROJECT_DOCUMENTATION.md` (earlier overview, written when the app was still called VetConnect)

Table of contents

1. The product in one page
2. Running it locally
3. Backend
4. Frontend
5. End-to-end flows
6. Rules and numbers worth remembering
7. Testing
8. Known gaps and next steps
9. Troubleshooting
10. Appendix: all API endpoints

---

1. The product in one page

1.1 Who uses it (roles)

Role (`Role` enum)	Who	What they do
`ROLE_CUSTOMER`	Pet owner	Adds pets, finds vets, books and pays for visits and pet-care services, sees visits, cancels, rates, scans pet QR codes
`ROLE_DOCTOR`	Veterinarian	Registers with credentials and documents, waits for admin approval, manages weekly hours, blocked slots and leave days, sees bookings, completes visits with notes and prescriptions, has a wallet
`ROLE_PROVIDER`	Pet-care business	Registers with a trade licence and a partner vet, waits for approval, manages services and prices, hours and capacity, sees bookings, has a wallet
`ROLE_ADMIN`	Platform staff	Reviews doctors and pet-care businesses (with their uploaded documents), approves or rejects with a reason, sees platform numbers

Vets and pet-care businesses are **separate accounts with separate screens**. A vet account has no Pet Care tab.

1.2 What the app does

- **Accounts:** the **mobile number** is the login (email is optional). Sign-up is confirmed with a one-time code (OTP). Passwords can be reset. Accounts can be deleted from the app.
- **Vet booking:** pick a vet, pick a free slot, pay the advance, and the slot is **confirmed at once** (no doctor approval).
- **Pet-care booking:** the same idea for grooming, boarding and training. Paying the advance confirms it and the business is notified.
- **Money:** advances go through a payment gateway (simulated for development, Razorpay for real). Every payment and refund also writes a row in a **wallet ledger** for the doctor or business.
- **Pets:** profile with photo, automatic unique **pet ID number**, a **QR code** per pet, a public page that shows the owner's contact when the QR is scanned, and a scanner inside the app.
- **Location:** country/state/district pickers, postal-code lookup, GPS, map pins, distance-based vet search. Clinics and pet-care businesses must pin their exact location.
- **Trust:** doctors and businesses upload documents, an admin checks them, and approved accounts show a small **Verified** badge.
- **Everything else:** in-app notifications, reviews and ratings, a guided tour, light and dark themes, input validation that blocks emoji and bad characters.

---

2. Running it locally

2.1 Prerequisites

Java 21, Node.js, PostgreSQL 16 (or Docker), and Expo Go on a phone (or a browser for the web build).

2.2 Database

Either create a local database once:

```
CREATE DATABASE vetconnect;
```

or start the one in Docker: `docker compose up -d` (service `postgres`, database `vetconnect`, host port `5433` by default).

2.3 Backend

```
cd backend
./mvnw spring-boot:run          # Windows PowerShell: .\mvnw.cmd spring-boot:run
```

Flyway creates and fills the schema on start (V1 to V22). Swagger UI is at `http://localhost:8080/swagger-ui.html`.

Useful environment variables (all have defaults in `application.yml`):

Variable	Meaning	Default
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`	PostgreSQL connection	`localhost`, `5432`, `vetconnect`, `postgres`, `postgres`
`PAYMENT_GATEWAY`	`simulated` or `razorpay`	`simulated`
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`	Razorpay keys (use `rzp_test_` keys to try)	empty
`CARE_CURRENCY`, `CARE_CURRENCY_SYMBOL`	Currency shown and charged	`USD`, `$` (use `INR` and `₹` for India)
`CLINIC_ADVANCE_PERCENT`	Share of a vet's fee paid as advance	`50`

2.4 Frontend

```
cd frontend
npm install
npx expo start            # add --clear after installing packages or if screens look stale
```

Press `w` for the browser, or scan the QR code with Expo Go (phone and PC on the same Wi-Fi). Set `EXPO_PUBLIC_API_URL` if the backend is not on the PC's default address. `npm run typecheck` runs `tsc --noEmit`.

2.5 Demo accounts (seeded by `DataInitializer`)

Sign-in is by mobile number. The login screen has quick-fill chips for these:

Role	Mobile	Password
Customer (John Doe)	`+1 555-0201`	`Customer@1234`
Doctor (Dr. Sarah Jenkins)	`+1 555-0111`	`Doctor@1234`
Admin	`+1 555-0100`	`Admin@1234`
Pet-care business (Happy Paws Grooming)	`+1 555-0301`	`Groomer@1234`

These are development credentials. Remove the seed data and rotate the JWT secret before any real deployment.

2.5.1 Where the one-time code goes

No SMS provider is connected. When someone signs up or asks for a reset, the code is **written to the backend log** as `[SIMULATED SMS] Verification code for ...` in `backend/logs/jenni.log`. Read the code from there while testing.

---

3. Backend

3.1 Layers

```
HTTP request
   |
   v
Filters:  RequestLoggingFilter  →  JwtAuthFilter (reads the Bearer token, loads the user)
   |
   v
controller/   (thin: URL, role check with @PreAuthorize, @Valid body, calls one service)
   |
   v
service/      (all business rules, @Transactional)
   |
   +──→ repository/ (Spring Data JPA)  ──→  PostgreSQL (schema owned by Flyway)
   +──→ payment/PaymentGateway (simulated or Razorpay)
   +──→ geo/GeocoderClient (Nominatim)
   +──→ NotificationService (writes in-app notifications)
   |
   v
dto/ objects go back to the client wrapped in ApiResponse
```

Rules the code follows:

- **Controllers hold no business logic.** Rules live in services, which makes them unit-testable with mocks.
- **The database schema is owned by Flyway.** Hibernate runs with `ddl-auto: validate`, so an entity that does not match the schema stops the app from starting. **Never edit a migration that has already run**: add a new `V<n>__name.sql` instead (Flyway checks file checksums).
- **Every response uses the same envelope** (`common/ApiResponse`): `{ success, message, data, timestamp }`. Lists use `PageResponse` (`content`, `totalElements`, `page`, `size`).
- **Errors are turned into JSON in one place**: `common/GlobalExceptionHandler` maps `BadRequestException` (400), `UnauthorizedException` (403), `ResourceNotFoundException` (404), `BookingConflictException` (409), `TooManyRequestsException` (429), `ExternalServiceException` and validation failures to the same envelope.

3.2 Package map (`backend/src/main/java/com/vetconnect`)

Package	Contents
`config/`	`SecurityConfig` (routes and roles), `JwtAuthFilter`, `AppProperties` (all `vetconnect.*` settings), `CorsConfig`, `ClockConfig` (a `Clock` bean so time can be faked in tests), `JacksonConfig` (rejects emoji in any JSON text), `RequestLoggingFilter`, `OpenApiConfig`, `DataInitializer` (demo data)

Package	Contents
`common/`	`ApiResponse`, `PageResponse`, `GlobalExceptionHandler`, `TextRules` (emoji and hidden-character check), `PhoneNumbers` (E.164 normalising), `PayoutRules` (UPI/bank validation), `GeoUtils` (distance maths), `LocationMatch` (the area-matching rule), `exceptions/`, `security/` (`JwtTokenProvider`, `UserPrincipal`, `CustomUserDetailsService`)
`controller/`	19 controllers, listed in the appendix
`service/`	The business logic (see 3.5)
`repository/`	One interface per entity (Spring Data JPA)
`domain/entities/`	JPA entities (see 3.4)
`domain/enums/`	Status and type enums (see 3.4)
`dto/`	Request and response objects, grouped by area (`auth`, `appointment`, `care`, `doctor`, `customer`, `pet`, `medical`, `review`, `notification`, `wallet`, `admin`)
`payment/`	`PaymentGateway` interface, `SimulatedPaymentGateway`, `RazorpayPaymentGateway`
`geo/`	`GeoService` (countries, states, districts from bundled data), `NominatimClient` (OpenStreetMap geocoding), `GeocoderClient`

3.3 Security

- **Authentication:** stateless JWT. Login returns an access token (24 hours) and a refresh token (7 days, stored in the `refresh_tokens` table so it can be revoked). `JwtAuthFilter` reads `Authorization: Bearer ...` and loads the user. **A disabled or deleted account is rejected even with an unexpired token.**
- **Authorisation:** `SecurityConfig` plus `@PreAuthorize("hasRole( ... )")` on controller methods. `/api/admin/**` needs `ADMIN`, `/api/customers/**` needs `CUSTOMER` or `ADMIN`, and so on.
- **Public endpoints (no token):** the `/api/auth/*` calls, `GET /api/geo/**`, `GET /api/care/config`, `GET /api/specializations/**`, doctor reviews, `GET /api/pet-photos/{name}`, `GET /api/pet-finder/{number}` (the QR scan lookup), `GET /api/p/{number}` (the "download Jenni" page) and `GET /api/documents/{id}/file` (protected by a signed link instead).
- **Passwords:** hashed with BCrypt. Reset codes expire in 30 minutes and lock after 5 wrong tries.
- **Input safety:** `JacksonConfig` rejects any JSON string containing emoji, pictographic symbols or hidden/control characters. DTO fields carry `@Pattern`, `@Size`, `@NotBlank`. Uploaded files are identified by their **first bytes** (not their name or declared type).
- **Rate limits:** OTP resend cooldown (30 s), OTP and reset attempts (5), pet-owner lookups (10 an hour per user), QR scans (20 an hour per IP), geocoder calls (30 a minute per client).

3.4 Data model

Tables are created by the migrations. Main entities:

Entity	Purpose
`User`	Login record: mobile (`phone` , E.164), email (a placeholder is generated when none is given), password hash, `role` , `isActive` , `emailVerified` (set true when the mobile is verified)
`CustomerProfile`	Pet owner details and home location (city, state, country, latitude/longitude)
`DoctorProfile`	Vet details: degree, council, licence number, clinic name/address/location with pin, fee, years, payout details, `verificationStatus` , admin reason
`ProviderProfile`	Pet-care business: business name, categories, trade licence, partner vet, location with pin, hours, capacity, `advancePercent` (default 30), payout details, `verificationStatus`
`VerificationDocument`	One uploaded certificate: owner (`DOCTOR` or `PROVIDER` plus id), type, random stored name, content type, size
`Pet`	A pet, its owner, photo URL, and an automatic, immutable **ID number** (`999` plus 12 digits)
`Appointment`	A vet visit: customer, doctor, pet, date and time, status, amounts (total, advance, balance), booking reference (`VC-...`), reschedule columns (kept for old data)
`CareBooking` / `CareBookingItem`	A pet-care booking and its priced lines (the main service plus add-ons)
`CareService`	A service a business offers: category, price per pet size, duration or per-night
`ProviderHours`	A business's weekly opening hours
`DoctorAvailability`	A vet's weekly working hours and slot length
`DoctorBlockedSlot`	A slot the vet blocked (or a customer-booked walk-in)
`DoctorLeave`	A whole day the vet is away
`CarePayment`	One payment attempt for an appointment or a care booking (gateway order id, status, refunded amount)
`WalletTransaction`	The ledger: a `CREDIT` for each paid advance, a `DEBIT` for each refund
`MedicalRecord` , `Prescription`	What the vet wrote after a visit

Entity	Purpose
`Review`, `CareReview`	Ratings for vets and for pet-care businesses
`Notification`	In-app notification for one user
`EmailVerificationOtp`, `PasswordResetToken`, `RefreshToken`	Sign-up codes, reset codes, refresh tokens
`Specialization`	Vet specialities (many-to-many with doctors)

Enums (`domain/enums`):

Enum	Values
`Role`	`ROLE_CUSTOMER`, `ROLE_DOCTOR`, `ROLE_PROVIDER`, `ROLE_ADMIN`
`VerificationStatus`	`PENDING`, `APPROVED`, `REJECTED`, `RESUBMITTED`
`AppointmentStatus`	`AWAITING_PAYMENT`, `PENDING` (old data only), `CONFIRMED`, `REJECTED`, `CANCELLED`, `COMPLETED`, `NO_SHOW`, `EXPIRED`
`CareBookingStatus`	`AWAITING_PAYMENT`, `REQUESTED` (old data only), `CONFIRMED`, `COMPLETED`, `CANCELLED`, `REJECTED`, `EXPIRED`
`PaymentStatus`	`CREATED`, `PAID`, `FAILED`, `EXPIRED`, `REFUNDED`, `PARTIALLY_REFUNDED`
`CareCategory`	`GROOMING`, `BOARDING`, `TRAINING`
`PetSpecies`, `Gender`, `PetSize`	Species (`DOG`, `CAT`, `BIRD`, `RABBIT`, `OTHER`), gender, size (`SMALL`, `MEDIUM`, `LARGE`, `GIANT` , worked out from the pet's weight)
`PricingUnit`	`PER_SESSION`, `PER_NIGHT`
`PayoutMethod`	`UPI`, `BANK`
`WalletTxnType`	`CREDIT`, `DEBIT`
`NotificationType`	Booking and payment events, verification events, reviews, `GENERAL`

3.5 Services (where the rules live)

Service	Responsibility
`AuthService`	Register (creates an unverified user and profile), verify OTP, login, refresh, logout, forgot and reset password

Service	Responsibility
`EmailOtpService`	Issue and check the 6-digit sign-up code (10 minutes, 5 attempts, 30 s cooldown). Writes the "SMS" to the log
`CustomerService`	Customer profile read and update
`DoctorService`	Search vets (by place or radius), profile create/update (a pin is required), weekly availability, free slots for a date, blocked slots, leave days, resubmit after rejection
`ProviderService`	Pet-care profile, services, hours, search near a customer, resubmit
`AppointmentService`	Book, pay (settle), confirm payment, cancel, reject, complete, reschedule, cancellation preview, hold expiry job
`CareBookingService`	The same lifecycle for pet-care bookings, plus review
`CareAvailabilityService`	Free slots for grooming/training and free nights for boarding, using the business's capacity
`CarePaymentService`	Create and settle `CarePayment` rows, call the gateway, refund, write the wallet ledger
`WalletService`	Credit an advance, debit a refund, return a wallet with history
`AdminService`	Pending and resubmitted lists, approve/reject with a reason (notifies the applicant), dashboard numbers
`DocumentService`	Upload, list, delete verification documents; signed 5-minute links; open a file
`PetService`	Pet CRUD, ID number generation, photo upload/removal
`PetPhotoStore`	Stores pet photos (JPG/PNG, random names)
`PetLookupService`	"Find a pet's owner" by number or QR scan, with rate limits and a notification to the owner
`MedicalRecordService`, `ReviewService`, `NotificationService`	Records and prescriptions, vet reviews, notifications
`AccountService`	Delete account. Needs the password; refused for admins, while there are open bookings (the person must cancel or finish them first), and while a doctor's or business's wallet still holds unpaid money. Otherwise personal details are scrubbed, files and notifications removed, and records that other people depend on stay but point to "Deleted ..."

3.6 Scheduled jobs

Both run every 60 seconds (`@Scheduled(fixedDelay = 60_000)`):

- `AppointmentService.expireStaleHolds()` and `CareBookingService.expireStaleHolds()` : a booking in `AWAITING_PAYMENT` whose 15-minute hold has passed becomes `EXPIRED` , its payment is expired and the slot is free again.

3.7 Configuration (`application.yml` and `AppProperties`)

Settings live under the `vetconnect.*` prefix: `jwt` (secret, 24 h / 7 d), `otp` , `reset` , `location` (default radius 25 km, max 100), `geo` (Nominatim URL, timeout, rate), `care` (hold 15 min, cancellation window 24 h, max boarding nights 30, max advance booking 180 days, currency), `payments` (gateway, Razorpay), `clinic` (advance percent), `uploads` (folder, 5 MB, 8 files per person, link 5 min), `app` (store links for the QR landing page), `cors` .

3.8 Migrations (Flyway)

Version	What it does
V1, V2	Initial schema and sample data
V3 to V7	Password reset tokens, data fixes, location, OTP and review reasons, country/state
V8	Pet-care providers, bookings and payments
V9	Clinic advance payments
V10	Appointment reschedule requests (the old approve/decline flow)
V11, V12	Payout details for businesses and doctors, the wallet ledger
V13, V14	Mobile-number login and a fix for 10-digit numbers
V15, V16	Doctor blocked slots and customer details on them
V17	Verification credentials (degree, council, licence, partner vet)
V18	Verification documents
V19	Doctor leave days
V20	Pet ID numbers
V21	Paid bookings confirm themselves: waiting vet requests and pet-care requests become `CONFIRMED`
V22	Clears time-change requests that were still waiting

4. Frontend

4.1 Overview

```
frontend/
  App.tsx            Providers: SafeArea > ThemeProvider > Paper > Auth > Navigation > Tour
  index.ts           Entry point
  app.json           Expo config (name Jenni, scheme jenni, automatic light/dark)
  src/
    api/             One file per backend area + client.ts (axios)
    context/         AuthContext
    navigation/      One navigator per role + types + deep-link handler
    screens/         auth, customer, doctor, provider, admin, shared
    components/      common, appointment, care, doctor, payment, pet
    theme/           colors, themed styles, ThemeProvider, shared styles
    tour/            Guided tour
    utils/           helpers, validators, inputRules, petLink, maps, ...
    data/            Static lists (Indian banks, State Veterinary Councils)
```

4.2 Providers and app start (`App.tsx`)

1. Fetches `GET /api/care/config` (currency symbol, advance percent, payment gateway), waiting up to 3 seconds so the first screen already shows the right currency.
2. `ThemeProvider` decides light or dark (saved choice, otherwise the phone's setting) and sets the colours **before anything renders**.
3. `Themed` wraps the app in `PaperProvider` , `AuthProvider` , `NavigationContainer` and `TourProvider` . Changing the theme re-creates this part (keyed by the scheme) and restores the current screen with the saved navigation state.
4. `RootNavigator` decides what to show: the scanned-pet page (public), a loading view while the session is restored, the **auth stack** when signed out, or the **role navigator** when signed in.

4.3 Authentication on the client

- `context/AuthContext.tsx` holds the signed-in user and has `login` , `register` , `verifyOtp` , `logout` , and `updateUserData` . Tokens are saved with `expo-secure-store` (browser `localStorage` on the web) through the `storage` helper in `api/client.ts` .
- `api/client.ts` is an axios instance (15-second timeout). A **request interceptor** adds the Bearer token. A **response interceptor** catches `401` , calls `/auth/refresh` once with the refresh token, retries the request, and if refresh fails signs the user out.
- **Registration documents** picked before the account exists are queued (`utils/pendingDocuments.ts`) and uploaded right after the mobile number is

verified (inside `verifyOtp`).

4.4 Navigation

Navigator	Tabs (bottom bar)	Notable stack screens
Auth	none	Splash, Onboarding, Login, Register (step-by-step), Verify OTP, Forgot and Reset password
Customer	Home, Doctors, Visits, Pet Care, Pets, Profile	Doctor profile, Book appointment, Booking summary, Appointment payment, Appointment details, Add/Edit pet, Pet details, Add review, Notifications, Help, Care provider, Care book, Care payment, Care bookings, Care booking details, Care review, Find a pet's owner
Doctor	Today, Appointments, Calendar, Profile	Appointment details, Consultation notes, Notifications, Wallet, Day slots
Provider (pet care)	none (one home screen)	Setup, Bookings, Services, Service edit, Hours & capacity, Business profile, Wallet, Account
Admin	none	Dashboard, Doctor/provider verification

Other navigation pieces: `navigationRef.ts` (navigate from outside React), `DeepLinkHandler.tsx` (opens `jenni://` and pet QR links), and `types.ts` (route parameter types).

4.5 Screens by role

Auth: `SplashScreen` (animated paw intro), `OnboardingScreen`, `LoginScreen` (mobile + password, demo chips), `RegisterScreen` (the wizard: role cards, then account, location, role details, credentials, payment), `VerifyOtpScreen`, `ForgotPasswordScreen`, `ResetPasswordScreen`.

Customer: `HomeScreen`, `FindDoctorsScreen` (search, filters, change the searched place without changing the profile), `DoctorProfileScreen`, `BookAppointmentScreen` (pet, date, slot, reason), `AppointmentPaymentScreen`, `BookingSummaryScreen`, `AppointmentDetailsScreen` (move to another time, cancel), `MyAppointmentsScreen` (the **Visits** tab: Clinic visits and Pet care sections), `PetListScreen`, `AddEditPetScreen`, `PetDetailsScreen` (QR, ID number, records), `CareHomeScreen`, `CareProviderScreen`, `CareBookScreen`, `CarePaymentScreen`, `CareBookingsScreen`, `CareBookingDetailsScreen`, `CareReviewScreen`, `AddReviewScreen`, `NotificationsScreen`, `HelpSupportScreen`, `CustomerProfileScreen`.

Doctor: `DoctorDashboardScreen` (wallet, today's slots card, stats), `DoctorAppointmentsScreen` (Upcoming, Completed, Cancelled),

`DoctorAppointmentDetailsScreen`, `ConsultationNotesScreen` (diagnosis, notes, prescriptions), `AvailabilityManagementScreen` (weekly hours, breaks, leave days), `DoctorSlotsScreen` (block a slot, add a walk-in), `DoctorProfileScreen` (credentials, documents, verification state, edit and resubmit).

Pet-care business: `ProviderHomeScreen` (wallet, upcoming and completed counts, tiles), `ProviderBookingsScreen`, `ProviderServicesScreen`, `ProviderServiceEditScreen`, `ProviderHoursScreen`, `ProviderProfileScreen`, `ProviderSetupScreen`, `ProviderAccountScreen` (Sign out and Delete account live here).

Admin: `AdminDashboardScreen`, `DoctorVerificationScreen` (the applicant cards with documents, "how to verify" steps, approve/reject with reason).

Shared: `WalletScreen`, `FindPetOwnerScreen` (scan or type a pet number), `PetFoundScreen` (what a scan shows, no sign-in needed).

4.6 API layer (`src/api`)

`authApi`, `customerApi`, `doctorApi`, `appointmentApi`, `careApi` (providers, bookings, payments, provider-side calls), `petApi`, `medicalApi`, `reviewApi`, `notificationApi`, `walletApi`, `documentApi`, `adminApi`, `accountApi`, `geoApi`. Each file exports typed functions (and the TypeScript types of that area) that call `apiClient`. Screens never call axios directly.

4.7 Components worth knowing

- `common/`: `AppInput` (every text box: filters typing and checks on blur using the `kind` prop), `PhoneField`, `EmailField`, `SelectField` (searchable dropdown), `LocationPicker` (country/state/district, postal code, one "Set location on map" button, the map popup with "Use my current location", pin preview and confirmation tick), `OsmMap` (Leaflet in a WebView, an iframe on the web), `DateField`, `DocumentsPicker` / `DocumentsCard`, `StatusBadge`, `VerifiedBadge`, `ThemeToggle`, `DeleteAccountCard`, `KeyboardScroll`.
- `appointment/`: `VisitCard` (one visit or pet-care booking), `TimeSlotPicker` (hides slots already past for today), `RescheduleRequestCard`, `AppointmentCard`.
- `care/`: `ProviderForm` (sign-up and profile form for a business), `PayoutFields` (UPI or bank, with the bank dropdown), `ProviderStatusCard`, `BookingCard`, `ProviderCard`.
- `payment/`: `PayPanel`, `PaymentMethods`, `RazorpayCheckout`, `WalletCard`.
- `pet/`: `PetQr`, `PetScanner`, `PetAvatar`, `PetTile`, `PetCard`.

4.8 Theme system (light and dark)

- `theme/colors.ts` has two palettes and a single exported object `colors` that is **overwritten in place** when the scheme changes (`applyScheme`). Tokens include `primary`, `background`, `page`, `canvas`, `card`, `subtle`, `text`,

`textSecondary`, `border`, `ink` (headings: dark in light mode, white in dark mode), `iconBorder`, and tinted pairs for success, warning, danger and info.

- Styles are written with `themedStyles(() => ({ ... }))` (in `theme/themed.ts`) instead of `StyleSheet.create`. It is a lazy proxy that rebuilds the style object when the theme version changes. Lookup tables that read colours use `themedTable`.
- Dark mode shows a near-black to plum gradient behind every screen (`Backdrop` in `App.tsx`). Screens use `colors.page` or `colors.canvas` (see-through in dark) as their background.
- `ThemeToggle` (sun/moon) sits at the top of the home screens. The choice is saved in secure storage.
- **Note:** after editing styles on a running dev server, restart Expo with `--clear` if the old look persists.

4.9 Input rules (`utils/inputRules.ts`)

`AppInput` takes a `kind` (for example `name`, `business`, `address`, `regNo`, `integer`, `decimal`, `phone`, `pin`, `otp`, `ifsc`, `upi`, `account`, `petName`, `time`, `note`). Each kind says which characters may be typed (emoji and hidden characters are always removed), a maximum length, the keyboard type, and a check that runs when the person leaves the field. Registration, pet and profile forms also check these rules on submit. Backend rules repeat the emoji rule for every JSON text field.

4.10 Guided tour (`src/tour`)

`TourProvider` dims the screen, lights up the element a step points to, and shows a card. Targets are registered with `tour.ref('some-id')` on views. Steps per role are in `tours.ts`. The tour opens once after sign-in unless "Hide this on launch" is ticked, and can be replayed from the Profile screen (`TakeTourButton`).

4.11 Utilities and data

File	Purpose
`utils/helpers.ts`	Money and currency formatting, date/time helpers, `confirmAction` and `notify` (work on both phone and web), error message extraction
`utils/validators.ts`	Email, password and phone rules (mirror the backend)
`utils/petLink.ts`	Builds pet QR links, parses scanned codes, the scanned-pet store
`utils/petPhoto.ts`	Picks and compresses pet photos
`utils/maps.ts`	Opens directions in the phone's map app

File	Purpose
`utils/guideStorage.ts`, `utils/pendingDocuments.ts`	Small storage helpers
`data/indianBanks.ts`	The bank dropdown list
`data/vetCouncils.ts`	All State Veterinary Councils plus VCI and Other

---

5. End-to-end flows

In each flow: **screen -> API call -> service -> database -> side effects**.

5.1 Sign-up and mobile verification (pet owner)

```
RegisterScreen (role: Pet Owner, 3 steps)
  step 1  choose role           step 2  name, mobile, password
  step 3  state + district (country is India by default), optional pin
        |  Create Account
        v
POST /api/auth/register      AuthService.register
  - mobile normalised to +91 ...  (PhoneNumbers); duplicate mobile --> 400
  - creates User (emailVerified=false), CustomerProfile
  - EmailOtpService.issue: 6-digit code, valid 10 min, written to jenni.log
        v
VerifyOtpScreen  -- types the code --->  POST /api/auth/verify-otp
  - code checked (5 tries), User.emailVerified=true
  - returns access + refresh tokens
        v
AuthContext stores tokens --> RootNavigator shows the Customer app
  (isNewRegistration=true --> the guided tour starts)
```

"Resend" calls `POST /api/auth/resend-otp` (30-second cooldown, identical answer for unknown numbers so nothing leaks).

5.2 Vet or pet-care registration, documents and admin approval

```
RegisterScreen (role: Veterinarian = 5 steps, Pet Care = 5 steps)
  account → clinic/business details → location + MAP PIN (mandatory, with the
  pin shown on a map and a confirmation tick) → credentials + documents
  → payment details (UPI or bank, bank from a dropdown)
        v
POST /api/auth/register  (server also requires the pin for both roles)
  creates User + DoctorProfile / ProviderProfile with verificationStatus = PENDING
        v
verify OTP (as 5.1) → queued documents uploaded:  POST /api/documents (multipart)
  DocumentService: sniff PDF/JPG/PNG, max 5 MB, max 8, random stored name
        v
The applicant sees "Waiting for approval". They are HIDDEN from customers.
        v
Admin: AdminDashboardScreen → DoctorVerificationScreen
  GET /api/admin/doctors/pending, /providers/pending
  opens each document (signed 5-minute link: GET /api/documents/{id}/link)
  PATCH /api/admin/doctors/{id}/verification  { approve | reject, reason }
        v
AdminService notifies the applicant.
  APPROVED  → visible to customers, shows the small "Verified" badge
  REJECTED  → applicant may edit details and upload new documents
              → POST /api/doctors/profile/resubmit (or /provider/profile/resubmit)
              → status RESUBMITTED → appears in the admin's "re-approval" list
```

Documents can be changed only while `PENDING` (during sign-up) or `REJECTED`, so the admin always reviews what they were shown.

5.3 Login, session and sign-out

```
LoginScreen → POST /api/auth/login { mobile, password }
   AuthService.login: password check, must be verified and active
   → access token (24 h) + refresh token (7 d, stored in DB)
Every request: interceptor adds the Bearer token.
On 401: one call to POST /api/auth/refresh, retry; if that fails → signed out.
Sign out: POST /api/auth/logout (revokes refresh tokens) and clears storage.
Forgot password: POST /api/auth/forgot-password → code in the log →
   POST /api/auth/reset-password (30 min, 5 tries).
Delete account: POST /api/account/delete { password } → refused if there are open
   bookings or unpaid wallet money; otherwise AccountService scrubs personal data,
   deletes files, notifications and refresh tokens; the token stops working at once.
```

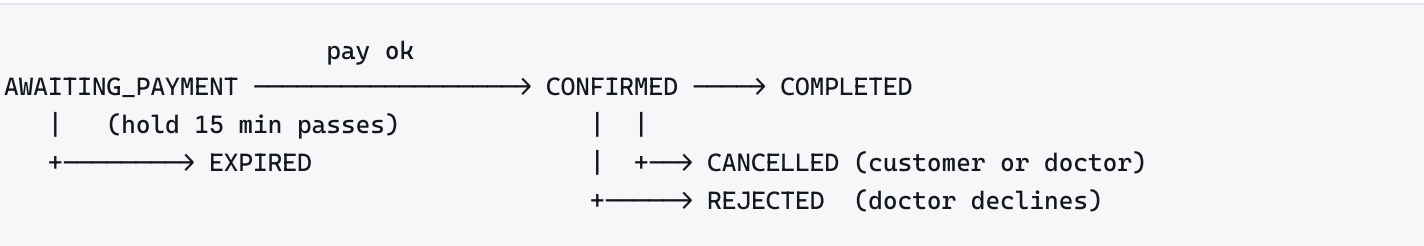
5.4 Finding a vet

```
FindDoctorsScreen → GET /api/doctors?… (radius, text, speciality, kind chips)
  DoctorService.searchDoctors: only APPROVED doctors
  the area rule (LocationMatch): if both sides have a pin, decide by distance
  (default 25 km, max 100); otherwise the place names must agree.
"Change" place: the screen sends city/state/country/lat/lng as parameters; the
  customer's saved profile is never changed.
```

5.5 Booking a vet appointment (instant confirmation)

```
DoctorProfileScreen → BookAppointmentScreen
  GET /api/doctors/{id}/availability?date=…   → free slots
  (weekly hours minus breaks, blocked slots, leave days, booked slots,
   and for today only slots that have not already passed)
        |  Book
        v
POST /api/appointments   AppointmentService.bookAppointment
  - checks: doctor approved, pet belongs to the customer, slot free
  - computes fee, advance (default 50%), balance
  - status = AWAITING_PAYMENT, slot HELD for 15 minutes (expiresAt)
  - CarePaymentService.createAdvance (gateway order created)
        v
AppointmentPaymentScreen (pay)
  simulated:  POST /api/appointments/{id}/payment/simulate {SUCCESS|FAILURE}
  Razorpay:   RazorpayCheckout → POST /api/appointments/{id}/payment/confirm
              { orderId, paymentId, signature }  (signature verified with the secret)
        v
AppointmentService.afterPaid
  - Appointment.status = CONFIRMED      ← no doctor approval step
  - CarePayment PAID, WalletService.creditAdvance (doctor's wallet CREDIT)
  - notification to the customer "Appointment Confirmed"
  - notification to the doctor "New Appointment Booked"
        v
BookingSummaryScreen: "You are booked!"
```

State machine:



A failed payment keeps the hold; `POST …/payment/retry` creates a new attempt while the 15 minutes last.

5.6 Cancelling, refunds and changing the time

```
Customer cancels (Visits tab or details):
  GET  /api/appointments/{id}/cancellation-preview  → message + refund amount
  PATCH /api/appointments/{id}/cancel
  rule: refund the advance in FULL if the visit is at least 24 hours away,
        otherwise the advance is kept.
Doctor rejects or cancels: always a FULL refund.
Refund = CarePaymentService.refund → gateway.refund → payment status
  REFUNDED/PARTIALLY_REFUNDED → WalletService.debitRefund (DEBIT row).
Change the time (customer): PATCH /api/appointments/{id}/reschedule
  - the new slot must be free; the appointment MOVES AT ONCE (no approval)
  - the doctor gets an "Appointment moved" notification
  (approve/reject/withdraw endpoints remain only for requests created before
   this rule; migration V22 cleared any still open)
```

5.7 The doctor's day

```
DoctorDashboardScreen: wallet card, "My day" slots card, stats (Today, Upcoming,
   Completed this month, Fees this month).
AvailabilityManagementScreen: PUT /api/doctors/availability (weekly hours, breaks)
   + leave days: POST/DELETE /api/doctors/me/leave
DoctorSlotsScreen: GET /api/doctors/me/schedule (booked / free / blocked),
   POST /api/doctors/me/blocked-slots (block time or add a walk-in with name and
   phone), DELETE /api/doctors/me/blocked-slots/{id}. Booked slots are locked.
After the visit: PATCH /api/appointments/{id}/complete, then
   ConsultationNotesScreen: POST /api/pets/{petId}/medical-records (diagnosis,
   notes, treatment, weight, prescriptions). The owner sees them on the pet.
Reviews: the owner rates the vet after a completed visit
   (POST /api/doctors/{id}/reviews); the doctor's average rating updates.
```

5.8 Pet-care booking (grooming, boarding, training)

```
CareHomeScreen → GET /api/care/providers?category= … &radius …  (approved
   businesses near the customer) → CareProviderScreen (services, hours, reviews)
CareBookScreen:
   grooming/training: GET /api/care/providers/{id}/slots?serviceId&date&petSize
   boarding:          GET /api/care/providers/{id}/boarding-availability
   (CareAvailabilityService uses the business's capacity and hours)
        v
POST /api/care/bookings   CareBookingService.create
   price by pet size, items (main service + add-ons), advance = business's % (30
   by default), status AWAITING_PAYMENT, 15-minute hold
        v
CarePaymentScreen → pay (simulate or Razorpay confirm, same as 5.5)
        v
afterPaid: status CONFIRMED, wallet credit, notification "New pet-care booking"
   to the business and "Booking confirmed" to the customer.
Cancel: customer → refund in full if at least 24 hours before the start,
   else none. The business can cancel a confirmed booking with a reason and the
   customer always gets a full refund (POST /api/provider/bookings/{id}/cancel).
Complete: the business marks it done on the day
   (POST /api/provider/bookings/{id}/complete) and the customer can review it
   (POST /api/care/bookings/{id}/review).
```

State machine:

```
AWAITING_PAYMENT --pay--> CONFIRMED --> COMPLETED
      |                       |
      +--> EXPIRED            +--> CANCELLED (customer or business)
```

5.9 Where bookings appear

- **Customer "Visits" tab:** `MyAppointmentsScreen` loads vet appointments and pet-care bookings for the chosen tab (Coming up, Past, Cancelled) and shows two sections: **Clinic visits** (top, with a "Next visit" card) and **Pet care**. Opening a card goes to the right details or payment screen.
- **Doctor:** Today dashboard and Appointments (Upcoming, Completed, Cancelled).
- **Business:** home counters (Upcoming, Completed) and the Bookings screen.

5.10 Payments and the wallet

```
PaymentGateway (interface)
   SimulatedPaymentGateway : development. Test buttons, no money moves.
   RazorpayPaymentGateway  : createOrder, verifySignature, refund via the API
CarePaymentService
   createAdvance → payment CREATED (+ gateway order)
   markPaid      → PAID + WalletService.creditAdvance
   markFailed / expire
   refund        → gateway.refund → refunded amount, status, WalletService.debitRefund
WalletService / WalletController: GET /api/wallet → balance + history
   (the wallet is a LEDGER: it shows what is owed to the doctor or business;
    nothing is paid out to a bank from it yet)
```

Security of payments: with Razorpay, a payment counts only when the **signature** returned by Razorpay is verified with the server's secret key, so forged or replayed results are refused. Card details never touch the app or the server.

5.11 Pets, ID numbers and QR codes

```
AddEditPetScreen → POST /api/pets
   PetService assigns a unique, unchangeable ID number (999 + 12 digits).
Photo: POST /api/pets/{id}/photo (multipart) → PetPhotoStore saves a random-named
   file; GET /api/pet-photos/{name} serves it (public, unguessable name).
PetDetailsScreen shows the QR (react-native-qrcode-svg) for
   <base>/p/<number>, with copy and "download as image".
Someone scans the QR:
   - phone has Jenni  → DeepLinkHandler → PetFoundScreen
                            → GET /api/pet-finder/{number} (no sign-in)
   - no Jenni         → GET /api/p/{number} returns an HTML page
                            "Download Jenni to view" with store links
   PetLookupService: 20 scans an hour per IP; the owner is notified of each lookup.
Signed-in users can also type the number or scan from FindPetOwnerScreen
   (GET /api/pets/lookup, 10 an hour per user).
The public page shows only what is needed to reach the owner (name, phone,
   pet's name and photo).
```

5.12 Notifications

`NotificationService.createNotification(user, title, message, type, reference)` is called by services when something happens (booking confirmed, cancelled, moved, payment received, approval decisions, reviews, pet lookups). The app reads them through `GET /api/notifications`, `/recent`, `/unread-count` and marks them read with `PATCH /api/notifications/{id}/read` or `/read-all`. The home screen bell shows a red dot when anything is unread. There is **no push notification service yet** (see section 8).

5.13 Location

- `GeoService` serves countries, states and districts from bundled data. `GET /api/geo/postal-code` and `/place` look a place up, and `/reverse` turns coordinates into a place using OpenStreetMap Nominatim (rate-limited and timed out).
- `LocationPicker` (frontend): the address boxes (state and district, India by default), a postal-code "Find", and a single **Set location on map** button. The popup has a small "Use my current location" button. If the phone's location is off it shows a popup that leads to the right settings screen. Tapping the map drops a pin; "Confirm location" fills the place from the pin.
- For vets and businesses the pin is **mandatory**, is shown on a map, and has to be confirmed with a tick. The server also refuses a vet or business without a pin, on sign-up and on every profile save.

5.14 Cross-cutting behaviours

- **Instant confirmation:** a paid booking is `CONFIRMED` immediately. Nothing waits for a doctor or a business.
- **Today's slots:** slots earlier than the current time are hidden when booking for today.
- **Verified badge:** once approved, a small "Verified" badge replaces the long status card for doctors and businesses.
- **Switching the sign-up role clears the form**, so data typed for one role never carries over to another.
- **Account screens:** the pet-care business has a separate Account screen for Sign out and Delete account.

---

6. Rules and numbers worth remembering

Rule	Value
Access token / refresh token	24 hours / 7 days
Sign-up code	6 digits, 10 minutes, 5 tries, 30 s between codes
Password reset code	30 minutes, 5 tries
Slot hold while paying	15 minutes
Vet advance	50 percent of the fee (`CLINIC_ADVANCE_PERCENT`)
Pet-care advance	the business's own percent, 30 by default
Customer cancellation	full refund if 24 hours or more before the visit, otherwise none
Doctor or business cancels / rejects	full refund
Search radius	default 25 km, max 100 km

Rule	Value
Boarding	at most 30 nights; bookings up to 180 days ahead
Uploads	PDF/JPG/PNG (documents) and JPG/PNG (pet photos), 5 MB each, 8 documents per account
Document links	signed, valid 5 minutes
Pet ID number	`999` plus 12 digits, unique, cannot be changed
Pet-owner lookups	10 an hour per user; QR scans 20 an hour per IP
Wallet	ledger only (credits and debits); no bank payout yet

---

7. Testing

- **Backend:** 255 JUnit tests (Mockito for services, plus a Spring context test). Run `cd backend && ./mvnw test`. They cover auth and OTP, appointments (payment, refunds, reschedule), pet-care bookings and availability, doctor slots and leave, documents, pets and photos, pet lookup, wallet, providers, text rules and the Jackson emoji check, geo and the Razorpay gateway.
- **Frontend:** `npm run typecheck` (TypeScript). There are no automated UI tests in the repository. Flows were checked manually and with throw-away browser scripts.
- **Manual check list:** register each role, verify the code from `jenni.log`, approve the new doctor or business as admin, book and pay as a customer (simulated gateway), cancel and check the refund and the wallet, scan a pet QR.

---

8. Known gaps and next steps

1. **SMS:** no provider is connected; codes go to the server log. Plug a real SMS service into `EmailOtpService`.
2. **Razorpay webhook:** not implemented. If a customer pays and the app closes before reporting back, that payment is not confirmed automatically (the hold expires and it must be refunded from the Razorpay dashboard). Add a webhook before going live.
3. **Payouts:** the wallet is a ledger only. Real payouts (for example Razorpay Route) are not built.
4. **Push notifications:** only in-app notifications exist.
5. **File storage:** uploads are saved on the server's disk under `backend/uploads/`, which is erased on most hosts at redeploy. Move to object storage such as S3 (all saving and reading

goes through `PetPhotoStore` and `DocumentService`) and back the folder up until then.

6. **QR landing page and links:** the "Download Jenni" page and universal links need a real domain and store URLs (`vetconnect.app.android-url`, `ios-url`).
7. **Unanswered or no-show handling:** there is no automatic rule for a visit that nobody completes.
8. **Old code paths:** the doctor reschedule approve/reject/withdraw endpoints and the `PENDING` / `REQUESTED` statuses remain only for old data and can be removed once no such rows exist.
9. **Naming:** some packages, the database name and older docs still say "vetconnect" (the app is Jenni).
10. **Security hardening before launch:** replace the seeded demo data, set a new JWT secret, restrict CORS, and review rate limits.

---

9. Troubleshooting

Symptom	Likely cause and fix
App shows old colours or a screen that was changed	Stale Metro cache. Stop Expo and run `npx expo start --clear` (always after installing packages).
"Cannot find module ..." on the phone after adding a package	Same: restart with `--clear`.
Backend will not start: Flyway checksum error	A migration that already ran was edited. Restore it and add a new migration instead.
Backend will not start: schema validation error	An entity changed without a migration (`ddl-auto: validate`). Add the migration.
No sign-up code arrives	Read it from `backend/logs/jenni.log` (`[SIMULATED SMS]`).
Prices show `$`	Start the backend with `CARE_CURRENCY=INR` and `CARE_CURRENCY_SYMBOL=₹`.
Phone cannot reach the backend	Same Wi-Fi, the PC's firewall, and `EXPO_PUBLIC_API_URL` set to the PC's address.
Scan from a phone fails	The PC's IP changed; regenerate the Expo QR code from the new address.
Location button does nothing	The phone's location is off or not allowed; the app shows a popup that opens the right settings.

---

10. Appendix: all API endpoints

All paths start with `/api`. "Public" means no token is needed.

Auth (`/auth`)

Method	Path	Purpose
POST	`/auth/register`	Create an account (public)
POST	`/auth/verify-otp`	Verify the mobile number, returns tokens (public)
POST	`/auth/resend-otp`	Send a new code (public)
POST	`/auth/login`	Sign in (public)
POST	`/auth/refresh`	New access token (public)
POST	`/auth/logout`	Sign out
POST	`/auth/forgot-password`, `/auth/reset-password`	Reset flow (public)

Account, customer, pets

Method	Path	Purpose
POST	`/account/delete`	Delete my account (needs the password; refused with open bookings or unpaid wallet money)
GET, PUT	`/customers/me`	My profile
GET, POST	`/pets`	My pets, add a pet
GET	`/pets/lookup`	Find a pet's owner by number
GET, PUT, DELETE	`/pets/{petId}`	One pet
POST, DELETE	`/pets/{petId}/photo`	Pet photo
GET	`/pet-photos/{name}`	Photo file (public)
GET	`/pet-finder/{number}`	QR scan lookup (public, rate-limited)

Method	Path	Purpose
GET	`/p/{number}`	"Download Jenni" HTML page (public)
GET, POST	`/pets/{petId}/medical-records`	Records (vet writes)
GET	`/pets/{petId}/prescriptions`	Prescriptions

Doctors and appointments

Method	Path	Purpose
GET	`/doctors`	Search vets
GET	`/doctors/{id}` , `/doctors/{id}/availability`	A vet, free slots for a date
GET, POST	`/doctors/{id}/reviews`	Reviews (GET is public)
GET, PUT, POST	`/doctors/profile/me` , `/doctors/profile`	My doctor profile, update, create
POST	`/doctors/profile/resubmit`	Ask for another review after rejection
PUT	`/doctors/availability`	Weekly hours
GET	`/doctors/appointments`	The doctor's appointments
GET	`/doctors/me/schedule`	Today's slots (booked, free, blocked)
POST, DELETE	`/doctors/me/blocked-slots` , `…/{blockId}`	Block or unblock a slot
GET, POST, DELETE	`/doctors/me/leave` , `…/{leaveId}`	Leave days
GET	`/specializations`	Speciality list (public)
POST	`/appointments`	Book (starts the hold)
GET	`/appointments/my` , `/appointments/{id}`	My visits, one visit
POST	`/appointments/{id}/payment/simulate` , `/confirm` , `/failed` , `/retry`	Payment steps
GET	`/appointments/{id}/cancellation-preview`	What a cancel would refund

Method	Path	Purpose
PATCH	`/appointments/{id}/cancel`, `/reject`, `/complete`, `/confirm`	Status changes (`confirm` is for old waiting rows)
PATCH	`/appointments/{id}/reschedule`	Move to a free slot (instant)
PATCH	`/appointments/{id}/reschedule/approve`, `/reject`, `/withdraw`	Only for requests made before the instant rule

Pet care

Method	Path	Purpose
GET	`/care/config`	Currency, advance %, gateway (public)
GET	`/care/providers`, `/care/providers/{id}`	Search and view businesses
GET	`/care/providers/{id}/slots`, `/boarding-availability`, `/reviews`	Availability and reviews
POST	`/care/bookings`	Book (starts the hold)
GET	`/care/bookings/my`, `/care/bookings/{id}`	My bookings
POST	`/care/bookings/{id}/payment/simulate`, `/confirm`, `/failed`, `/retry`	Payment steps
GET	`/care/bookings/{id}/cancellation-preview`	Refund preview
POST	`/care/bookings/{id}/cancel`, `/review`	Cancel, review
GET, POST, PUT	`/provider/profile/me`, `/provider/profile`	The business profile
PUT	`/provider/hours`	Weekly hours and capacity
POST	`/provider/profile/resubmit`	Ask for another review
GET, POST, PUT, DELETE	`/provider/services`, `… /{serviceId}`	The business's menu
GET	`/provider/bookings`	Bookings by group

Method	Path	Purpose
POST	`/provider/bookings/{id}/accept`, `/reject`, `/cancel`, `/complete`	Provider actions (`accept` and `reject` apply only to old waiting rows)

Admin, documents, wallet, notifications, geo

Method	Path	Purpose
GET	`/admin/dashboard`	Platform numbers
GET	`/admin/doctors/pending`, `/resubmitted`, `/providers/pending`, `/resubmitted`	Review queues
PATCH	`/admin/doctors/{id}/verification`, `/admin/providers/{id}/verification`	Approve or reject with a reason
GET	`/admin/appointments`	All appointments
GET	`/admin/documents/{ownerType}/{ownerId}`	An applicant's documents
POST, GET	`/documents`	Upload (multipart), list mine
DELETE	`/documents/{id}`	Remove a document (only while pending or rejected)
GET	`/documents/{id}/link`	Signed 5-minute link
GET	`/documents/{id}/file?exp=&sig=`	The file (public, signature-protected)
GET	`/wallet`	Wallet balance and history
GET	`/notifications`, `/recent`, `/unread-count`	Read notifications
PATCH	`/notifications/{id}/read`, `/read-all`	Mark read
GET	`/geo/countries`, `/countries/{c}/states`, `/countries/{c}/states/{s}/districts`	Place lists (public)
GET	`/geo/postal-code`, `/geo/place`, `/geo/reverse`	Place lookups (public, rate-limited)